CORRIGÉ · PROBLÈME · ANNALES SPÉ

Couverture minimale par des chaînes

Énoncé

Langage OCaml exclusivement. On représente une relation sur $V = \{v_0, \dots, v_{n-1}\}$ par une matrice `r : bool array array` de taille $n \times n$, avec `r.(i).(j)` vrai si et seulement si $v_i \trianglelefteq v_j$ (`type order = bool array array`).

Une couverture est une famille $(C_\ell)_{1 \leqslant \ell \leqslant \lambda}$ de chaînes dont la réunion est V.

1. Si (C_ℓ) est une couverture et A une antichaîne de cardinal α, montrer que $\alpha \leqslant \lambda$.

Graphe des poursuites : graphe biparti P sur $W_0 = \{w_i\}$ et $W_1 = \{w_{n+i}\}$ (deux copies de V), avec une arête (w_i, w_{n+j}) dès que $i \neq j$ et $v_i \trianglelefteq v_j$. Représentation : `type bipartite = int list array` de longueur $2n$, chaque arête notée deux fois. Un couplage est un ensemble d'arêtes d'extrémités distinctes.

1. Écrire `bipartite_of_order (r:order) : bipartite`.
2. Pour un couplage M, on prend la partition (C_ℓ) de V la plus fine possible telle que $(w_i, w_{n+j}) \in M$ impose que v_i et v_j soient dans la même part. Montrer que chaque C_ℓ est une chaîne.

Un couplage est codé par `m : int array` de longueur n : `m.(j) = i` si $(w_i, w_{n+j}) \in M$, et -1 si w_{n+j} n'est pas couvert.

1. Écrire `chains_of_matching (m:int array) : int list list`.
2. Une couverture par des chaînes disjointes étant fixée, construire un couplage qui la redonne par la question 19.
3. Montrer que la construction de 19 donne une couverture par un nombre minimum de chaînes si et seulement si M est de cardinal maximum.
4. Écrire `maximum_matching (p:bipartite) : int array`.
5. Écrire `minimum_cover (r:order) : int list list`.
6. Complexité de `minimum_cover` en fonction de n.

Décoder l'énoncé

Ce que l'énoncé te donne.

- Dans un couplage, chaque w_i a au plus un partenaire « à droite » et chaque w_{n+j} au plus un « à gauche » : on construit des chemins.

Ce qu'on te demande. Calculer le nombre minimal de chaînes qui couvrent un ordre, grâce à un couplage maximum (le théorème de Dilworth en version algorithmique).

Corrigé

Question 17

Étape 1. Q17. Antichaîne et couverture

Chaque élément $a \in A$ appartient à au moins une chaîne de la couverture ; on en choisit une, notée $C_{\ell(a)}$. L'application $a \mapsto \ell(a)$ va de A dans $[\![1, \lambda]\!]$.

Elle est **injective** : si $\ell(a) = \ell(a')$ avec $a \neq a'$, alors a et a' sont dans la même chaîne, donc comparables, ce qui contredit le fait que A est une antichaîne.

$$\alpha = \operatorname{card} A \leqslant \operatorname{card}[\![1, \lambda]\!] = \lambda.$$

Question 18

Étape 1. Q18. Le graphe des poursuites

```
let bipartite_of_order (r:order) : bipartite =
  let n = Array.length r in
  let p = Array.make (2 * n) [] in
  for i = 0 to n - 1 do
    for j = 0 to n - 1 do
      if i <> j && r.(i).(j) then begin
        p.(i) <- (n + j) :: p.(i);       (* arête w_i -- w_{n+j} vue de w_i *)
        p.(n + j) <- i :: p.(n + j)      (* et vue de w_{n+j} *)
      end
    done
  done;
  p
```

Complexité $O(n^2)$.

Question 19

Étape 1. Q19. Chaque part est une chaîne

Graphe des liens : on relie $v_i \to v_j$ quand $(w_i, w_{n+j}) \in M$. Comme M est un couplage :

- w_i est extrémité d'au plus une arête : chaque v_i a au plus un **successeur** ;
- w_{n+j} est extrémité d'au plus une arête : chaque v_j a au plus un **prédécesseur**.

La partition la plus fine qui met dans la même part les extrémités de chaque lien (c'est celle qui maximise λ) est celle des composantes connexes de ce graphe. Chaque composante est donc un chemin ou un cycle.

Pas de cycle : un cycle $v_{i_1} \to v_{i_2} \to \ldots \to v_{i_p} \to v_{i_1}$ ($p \geqslant 2$, sommets distincts) donnerait par transitivité $v_{i_1} \trianglelefteq v_{i_2}$ et $v_{i_2} \trianglelefteq v_{i_1}$, donc $v_{i_1} = v_{i_2}$ par antisymétrie, ce qui est absurde.

Conclusion : chaque part est un chemin $v_{i_1} \to v_{i_2} \to \ldots \to v_{i_p}$ avec $v_{i_1} \trianglelefteq v_{i_2} \trianglelefteq \cdots \trianglelefteq v_{i_p}$. Par transitivité, deux éléments quelconques de la part sont comparables : **chaque C_ℓ est une chaîne**.

Question 20

Étape 1. Q20. Des chaînes à partir du couplage

`m.(j) = i` signifie que v_i a pour successeur v_j. Les sommets sans prédécesseur (`m.(j) = -1`) sont les débuts des chaînes ; on suit les successeurs.

```
let chains_of_matching (m:int array) : int list list =
  let n = Array.length m in
  let succ = Array.make n (-1) in
  Array.iteri (fun j i -> if i >= 0 then succ.(i) <- j) m;   (* lien v_i -> v_j *)
  let rec follow v = if v = -1 then [] else v :: follow succ.(v) in
  let chains = ref [] in
  for j = 0 to n - 1 do
    if m.(j) = -1 then chains := follow j :: !chains     (* v_j débute une chaîne *)
  done;
  !chains
```

Chaque sommet est dans exactement un chemin (Q19), et chaque chemin a exactement un début : on obtient toutes les chaînes, une seule fois chacune. Complexité $O(n)$.

Question 21

Étape 1. Q21. De la couverture au couplage

Soit une couverture par des chaînes disjointes. On range chaque chaîne C_ℓ dans l'ordre croissant : $c_1 \trianglelefteq c_2 \trianglelefteq \cdots \trianglelefteq c_p$ (possible car tous ses éléments sont comparables). On met dans M les arêtes $(w_{c_t}, w_{n+c_{t+1}})$ pour $1 \leqslant t < p$, pour toutes les chaînes.

- Ce sont des arêtes du graphe des poursuites : $c_t \neq c_{t+1}$ et $v_{c_t} \trianglelefteq v_{c_{t+1}}$.
- C'est un couplage : dans une chaîne, chaque élément a au plus un suivant et au plus un précédent, et les chaînes sont disjointes.

- La construction de Q19 relie exactement les éléments consécutifs de chaque chaîne : ses parts sont les C_ℓ.

On a de plus $|M| = \sum_\ell (|C_\ell| - 1) = n - \lambda$.

Question 22

Étape 1. Q22. Minimum de chaînes et couplage maximum

Nombre de chaînes : dans Q19, les parts sont des chemins ; un chemin à p sommets contient $p-1$ liens. En sommant sur les parts : $n - \lambda = |M|$, soit

$$\lambda = n - |M|.$$

Couvertures disjointes : on peut toujours rendre une couverture disjointe sans augmenter son nombre de chaînes (on retire un élément des chaînes où il apparaît en double ; une partie d'une chaîne est une chaîne). Le minimum $\lambda_{\min}$ est donc atteint par une couverture disjointe, et par Q21 celle-ci provient d'un couplage de cardinal $n - \lambda_{\min}$.

Équivalence :

- si $|M|$ est maximum, alors $n - |M| \leqslant n - |M'|$ pour tout couplage M', en particulier pour celui de Q21 : $\lambda = n - |M| \leqslant \lambda_{\min}$, donc $\lambda = \lambda_{\min}$;
- réciproquement, si $\lambda = n - |M| = \lambda_{\min}$, un couplage plus grand M' donnerait une couverture de $n - |M'| < \lambda_{\min}$ chaînes, ce qui est impossible.

$$\lambda_{\min} = n - |M|_{\max}.$$

Question 23

Étape 1. Q23. Couplage maximum par chemins augmentants

Méthode (algorithme de Kuhn) : on traite les sommets w_i de W_0 un par un. Pour w_i, on cherche un voisin w_{n+j} non encore visité ; s'il est libre, on le couple à w_i ; sinon, on essaie de recaser son partenaire actuel `m.(j)` ailleurs, récursivement. Une réussite correspond à un chemin augmentant, qu'on inverse.

```
let maximum_matching (p:bipartite) : int array =
  let n = Array.length p / 2 in
  let m = Array.make n (-1) in              (* m.(j) = partenaire de w_{n+j} *)
  (* essaie d'attribuer un partenaire à w_i ; seen évite de reboucler *)
  let rec augment i seen =
    List.exists (fun w ->
      let j = w - n in
      if seen.(j) then false
      else begin
        seen.(j) <- true;
        if m.(j) = -1 || augment m.(j) seen then (m.(j) <- i; true)
```

```
        else false
      end) p.(i)
  in
  for i = 0 to n - 1 do
    ignore (augment i (Array.make n false))
  done;
  m
```

`List.exists` s'arrête au premier succès. **Correction** : chaque recherche réussie augmente $|M|$ de 1 ; quand plus aucune recherche ne réussit, il n'existe plus de chemin augmentant, et le couplage est maximum par le théorème de Berge.

Question 24

Étape 1. Q24. Couverture minimale

```
let minimum_cover (r:order) : int list list =
  chains_of_matching (maximum_matching (bipartite_of_order r))
```

D'après Q22, un couplage maximum donne une couverture par un nombre minimum de chaînes (disjointes).

Question 25

Étape 1. Q25. Complexité

- `bipartite_of_order` : $O(n^2)$.
- `maximum_matching` : n recherches ; chacune visite chaque w_{n+j} au plus une fois et parcourt au plus une fois chaque liste d'adjacence, soit $O(n + \text{nombre d'arêtes}) = O(n^2)$. Total $O(n^3)$.
- `chains_of_matching` : $O(n)$.

Complexité de `minimum_cover` : $O(n^3)$.

Le piège

En Q22, oublier de justifier qu'une couverture minimale peut être prise avec des chaînes disjointes : sans cela, Q21 ne s'applique pas.

La méthode

Couverture minimale par des chaînes (Dilworth) : graphe biparti « i peut précéder j », couplage maximum, puis $\lambda_{\min} = n - |M|$.

Rappels de cours utilisés

- **Couplage et chemins augmentants.** Un couplage est un ensemble d'arêtes deux à deux sans extrémité commune. Un chemin augmentant alterne arêtes hors couplage et arêtes du couplage entre deux sommets libres. **Théorème de Berge** : un couplage est maximum si et seulement s'il n'admet aucun chemin augmentant.